

DEVELOPER PROJECT REPORT

QT PDF Editor

A browser-based PDF editor, and an honest look at how it's built

Repository: [quantiradev/qt_pdf-editor](#) • Branch: sprint1

Report Date: July 13, 2026

Classification: Internal – Engineering Documentation

- The REST API is broad and consistent, covering the entire PDF lifecycle – CRUD, page operations, merge/split, form fields, watermarking, and multi-format export.

Where the Risk Sits

- No automated test suite yet – no unit tests, no integration tests, nothing end-to-end.
- Deployment today means running a Windows batch script that starts Uvicorn locally; there's no Dockerfile and no CI/CD pipeline.
- `auth.py` falls back to a hardcoded JWT secret if the environment variable isn't set, and passwords are hashed with only 1,000 PBKDF2 iterations – both worth fixing before this goes anywhere near the public internet.
- Metadata lives in a single `db.json` file guarded by an in-process lock, which is fine for one server and won't survive scaling out to more than one.

2. Project Overview & Objectives

2.1 The Problem

Editing a PDF properly usually means opening a desktop app like Acrobat, or routing the file through some conversion pipeline. QT PDF Editor's whole premise is to skip that: upload once, and everything after – viewing, marking up, restructuring – happens against that same file, in the browser, with edits written straight back into a standards-compliant PDF rather than bounced through an intermediate format.

2.2 What It Sets Out To Do

- Render PDFs faithfully in the browser, fast enough that scrolling and zooming actually feel responsive.
- Cover the annotation types people actually reach for: text, highlight/underline/strikeout, freehand ink, shapes, images, sticky notes, hyperlinks.
- Support real in-place text editing rather than an overlay box pretending to be one, by detecting paragraph structure and reflowing edited text with the source document's own fonts.
- Handle document management, not just markup: rename, soft/hard delete, restore, rotate/insert/duplicate/reorder/delete/extract pages, split, merge.
- Gate access behind accounts, and offer export to PDF, PNG, or JPEG with control over page range and DPI.
- Keep a real undo/redo history at the document level, so a bad edit doesn't mean starting over.

2.3 Who It's For

Individuals and small teams who need to mark up, redline, fill in, or lightly restructure a PDF, and would rather not install anything to do it.

3. System Architecture

The system splits along the usual client/server line, and it sticks to that split cleanly: the client owns rendering and interaction, the server owns the actual PDF bytes and every structural change made to them. Nothing that affects document correctness is duplicated on the client – if you want to know what a document actually looks like, the server's copy is the answer.

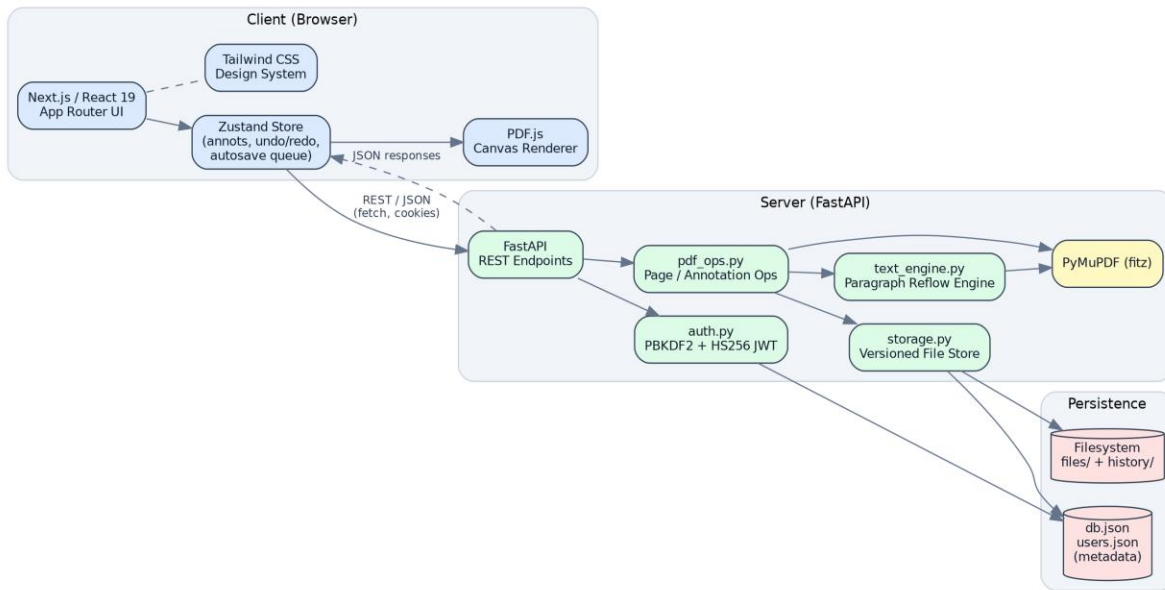


Figure 1 – QT PDF Editor system architecture

3.1 Frontend

Technology	Role
Next.js (React 19, App Router)	Routing for auth, editor, preview, and profile pages; the editor itself lives in client components
Zustand	Holds the editor's live state: the loaded document, annotation list, undo/redo stacks, tool options, panel layout, autosave queue
Tailwind CSS v4	Styling across the whole app, from the landing page to the editor chrome
PDF.js (pdfjs-dist)	Renders pages to <canvas> on the client – the only place the PDF actually gets parsed for display
Lucide React	The icon set used throughout the toolbar and sidebars

The editor itself (components/editor-studio/) is built around one Editor.tsx shell that pulls together a Toolbar, a LeftSidebar for pages and the outline, CanvasStage/PageView for the actual rendering, a RightSidebar for properties and the annotation list, an AnnotLayer overlay that handles interactive editing, and a StatusBar. A separate Previewer/PreviewTopBar pair handles the read-only view used for shared documents.

3.2 Backend

Technology	Role
FastAPI	The REST layer – routing, request validation via Pydantic, structured error handling